

Ch11. 온라인 쇼핑몰 데이터 요약하기

```
In [2]: import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image
```

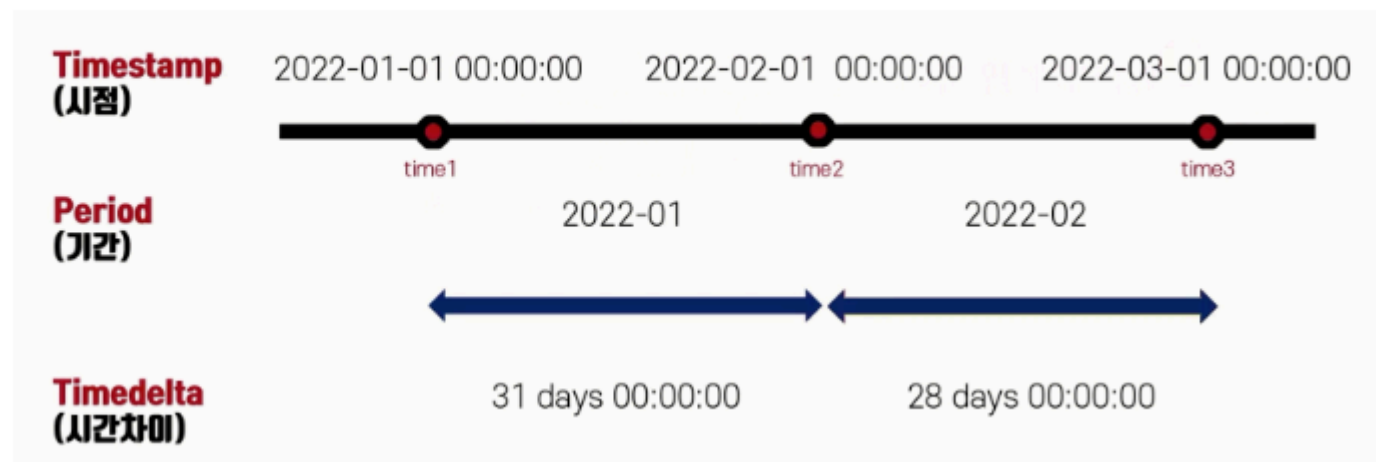
1. 시계열 데이터란?

시계열 데이터

- 날짜와 시간에 관련된 데이터
- 판다스는 시계열 데이터를 다루는 것에 장점이 있다

```
In [3]: Image('1.png')
```

Out[3]: 판다스의 세가지 시계열 데이터



```
In [4]: Image('2.png')
```

Out[4]: 시계열 데이터에서 **Timestamp**가 가장 중요한 이유

기간내 판매량을 기록한 데이터 프레임

	날짜	판매량
0	2022-01	20
1	2022-02	30
2	2022-03	50

	날짜	판매량
0	2022-01-01	20
1	2022-02-01	30
2	2022-03-01	50

기간이니 좌측과 같이 period를 사용해야 정확하겠지만
우측과 같이 해당 월별 첫째날의 timestamp를 대표로 기간을 나타내도 된다

시간 경과별 몸무게를 기록한 데이터프레임

	일차	몸무게
0	1 days	65.50
1	2 days	65.48
2	3 days	65.46

	일차	몸무게
0	1	65.50
1	2	65.48
2	3	65.46

시간의 경과를 좌측과 같이 timedelta로 나타내는 것이 정확하겠지만
우측처럼 숫자를 사용해도 된다.

-> 가장 중요한 것은 timestamp를 다루는 것이다

Timestamp(datetime)에서 배워야할 것

- datetime으로 변환(to_datetime)
- 주기
- DatetimeIndex 생성(date_range)
- dt 접근자
- 그룹집계(resample)

2. 시계열 데이터로 변환하기(to_datetime)

pandas to_datetime

자료형을 datetime으로 바꿔주는 함수

인자(parameter)

arg (datetime으로 변환 가능한 데이터)

datetime으로 변환할 데이터

errors (인수는 'ignore', 'raise', 'coerce' / 기본값 'raise')

에러 처리 방식

- 'raise' : 변환할 수 없는 객체를 만나면 에러를 일으킨다
- 'coerce' : 변환할 수 없는 객체를 만나면 해당 부분만 NaT으로 바꾸고 변환을 수행한다.
- 'ignore' : 변환할 수 없는 객체를 만나면 모두 변환하지 않고 그냥 input을 그대로 반환한다

format (str, 기본값 None)

문자열이 어떤 날짜형을 나타내는지 형식을 지정할 수 있다

예) 2023년 1월 3일이 '03/01/2023'으로 표현되어 있는 데이터라면 '%d/%m/%Y'로 표현한다

unit (str, 기본값은 'ns')

Timestamp가 숫자로 주어질 때 숫자의 기본단위. 기본값은 nano second

origin (scalar, 기본값 'unix')

Timestamp가 숫자로 주어질 때 기준 날짜. unit에서 정한 단위만큼의 숫자를 기준 날짜에서 더한다

예) origin으로 1970-01-01을 기준으로 정하고 unit가 D라면 숫자1은 기준일로부터 1일 지난 후인 1970-01-02를 의미한다

- 'unix' (or POSIX) : origin 은 1970-01-01.

- 'julian' : BC 4713년 1월 1일 정오. unit는 'D'여야 한다
- Timestamp 로 변환가능한 문자열을 넣어도 된다

to_datetime 함수 설명 : <https://kimpanda.tistory.com/105>

to_datetime 판다스 공식 문서 : https://pandas.pydata.org/docs/reference/api/pandas.to_datetime.html

Timestamp와 datetime은 어떤 차이가 있나요?

1640962800 → 숫자로 나타낸 Timestamp

2022-01-01 00:00:00 → datetime으로 나타낸 Timestamp

날짜형식 Timestamp의 자료형이 datetime이다

→ 숫자를 다루기 위한 단원은 아니기에 이번 단원의 Timestamp는 datetime이다

```
In [ ]: # 실습 준비코드
import pandas as pd
data = [['2023-01-01', 10], ['2023-01-02', 50], ['2023-02-02', 20],
        ['2023-Feb-03', 30], ['2023-Feb-04', 30], ['2023/Mar/01', 30]]
df = pd.DataFrame(data, columns=['날짜', '판매량'])
df
```

```
Out[ ]:
```

	날짜	판매량
0	2023-01-01	10
1	2023-01-02	50
2	2023-02-02	20
3	2023-Feb-03	30
4	2023-Feb-04	30
5	2023/Mar/01	30

```
In [ ]: # to_datetime 함수로 timestamp로 변경하기
pd.to_datetime(df['날짜'])
```

```
Out[ ]: 0    2023-01-01
1    2023-01-02
2    2023-02-02
3    2023-02-03
4    2023-02-04
5    2023-03-01
Name: 날짜, dtype: datetime64[ns]
```

```
In [ ]: # astype 함수로 timestamp로 변경하기 (잘 쓰지 않는다)
df['날짜'].astype('datetime64[ns]')
```

```
Out[ ]: 0    2023-01-01
1    2023-01-02
2    2023-02-02
3    2023-02-03
4    2023-02-04
5    2023-03-01
Name: 날짜, dtype: datetime64[ns]
```

```
In [ ]: # to_datetime 함수의 장점 (바꿀 수 없는 데이터는 NaT로 바꾼다)
s = pd.Series(['2022-01/03', '김판다'])
pd.to_datetime(s, errors='coerce')
```

```
Out[ ]: 0    2022-01-03
1           NaT
dtype: datetime64[ns]
```

시계열 데이터의 인덱싱과 슬라이싱(DatetimeIndex로 만들어야 한다)

```
In [ ]: # df를 복제한 df1으로 날짜열을 datetime으로 바꾸자
df1 = df.copy()
df1['날짜'] = pd.to_datetime(df1['날짜'])
```

```
In [ ]: # 자료형을 바꾼 날짜열을 index로 지정
df1 = df1.set_index('날짜')
df1
```

Out[]:

판매량	
날짜	
2023-01-01	10
2023-01-02	50
2023-02-02	20
2023-02-03	30
2023-02-04	30
2023-03-01	30

In []:

```
# 2023-02의 자료만 가져오기
df1.loc['2023-02'] # 단수의 문자열이라 df1['2023-02']을 사용하는 것보다 loc사용
```

Out[]:

판매량	
날짜	
2023-01-01	10
2023-01-02	50

In []:

```
# 2023-02 부터 자료를 슬라이싱해서 가져오기
df1.loc['2023-02':]
```

Out[]:

판매량	
날짜	
2023-02-02	20
2023-02-03	30
2023-02-04	30
2023-03-01	30

파일에서 데이터 프레임을 불러올 때 **datetime** 자료형으로 지정하기

`read_csv` 나 `read_excel` 의 `parse_dates` 인자에 해당 열을 리스트로 입력한다(단독이어도 리스트로 입력)

예) 시간열을 `datetime` 자료형으로 불러올 경우

```
parse_dates=['시간']
```

프로젝트 데이터

온라인 쇼핑몰 데이터 : https://raw.githubusercontent.com/panda-kim/csv_files/main/shop_data.csv

```
In [5]: # 프로젝트 코드
import pandas as pd
pd.options.display.max_rows = 6 # 판다스 1.4+ 에서 6행까지만 출력
url = 'shop_data.csv'
#url = 'https://raw.githubusercontent.com/panda-kim/csv_files/main/shop_data.csv'
```

```
In [6]: # csv파일에서 불러올 때 datetime으로 지정하고 index로 지정하기
df_ex1 = pd.read_csv(url, parse_dates=['시간'], index_col=0)
df_ex1
```

Out[6]:

	유저코드	카테고리	제품코드	가격	행동
시간					
2022-06-01 00:00:01	u514028527	패션	p1307115	411.59	V
2022-06-01 00:00:13	u550193582	화장품	p12705151	51.22	V
2022-06-01 00:00:13	u551977964	식품	p4100249	583.80	V
...	...	...	...	...	...
2022-06-07 23:59:47	u566067205	인테리어	p16300035	31.92	V
2022-06-07 23:59:56	u553835872	가전	p1005192	205.67	V
2022-06-07 23:59:59	u518620512	식품	p3600661	306.05	V

81433 rows × 5 columns

```
In [ ]: # 2022년 6월 1일 데이터만 인덱싱
df_ex1.loc['2022-06-01']
```

Out[]:

	유저코드	카테고리	제품코드	가격	행동
시간					
2022-06-01 00:00:01	u514028527	패션	p1307115	411.59	V
2022-06-01 00:00:13	u550193582	화장품	p12705151	51.22	V
2022-06-01 00:00:13	u551977964	식품	p4100249	583.80	V
...	...	...	...	...	...
2022-06-01 23:59:45	u566215254	화장품	p1004341	199.84	V
2022-06-01 23:59:47	u512604876	컴퓨터	p11600330	23.14	V
2022-06-01 23:59:56	u533241399	식품	p1004386	776.08	V

16974 rows × 5 columns


```
In [ ]: # 2022년 6월 1일 23시의 데이터만 인덱싱
df_ex1.loc['2022-06-01 23']
```

```
Out[ ]:
```

	유저코드	카테고리	제품코드	가격	행동
시간					
2022-06-01 23:00:01	u512686201	컴퓨터	p1005073	1133.09	V
2022-06-01 23:00:01	u546852746	식품	p15100148	378.36	V
2022-06-01 23:00:05	u545634079	화장품	p1005235	437.51	V
...	...	...	...	...	...
2022-06-01 23:59:45	u566215254	화장품	p1004341	199.84	V
2022-06-01 23:59:47	u512604876	컴퓨터	p11600330	23.14	V
2022-06-01 23:59:56	u533241399	식품	p1004386	776.08	V

830 rows × 5 columns

```
In [ ]: # 2022년 6월 1일 23시 00분의 데이터만 인덱싱
df_ex1.loc['2022-06-01 23:00']
```

Out[]:

	유저코드	카테고리	제품코드	가격	행동
시간					
2022-06-01 23:00:01	u512686201	컴퓨터	p1005073	1133.09	V
2022-06-01 23:00:01	u546852746	식품	p15100148	378.36	V
2022-06-01 23:00:05	u545634079	화장품	p1005235	437.51	V
...	...	...	...	...	...
2022-06-01 23:00:52	u538139300	식품	p12708848	128.70	V
2022-06-01 23:00:54	u513346578	식품	p14700312	317.61	V
2022-06-01 23:00:54	u561817477	식품	p10800025	54.00	V

12 rows × 5 columns

In []:

```
# 2022-06-02 부터 2022-06-04까지 슬라이싱
df_ex1.loc['2022-06-02':'2022-06-04']
```

Out[]:

	유저코드	카테고리	제품코드	가격	행동
시간					
2022-06-02 00:00:02	u566601875	인테리어	p54900002	46.33	V
2022-06-02 00:00:10	u513357826	식품	p1701432	483.90	V
2022-06-02 00:00:19	u518692140	자동차	p1306586	630.39	V
...	...	...	...	...	...
2022-06-04 23:59:31	u567495419	인테리어	p1005233	149.04	V
2022-06-04 23:59:35	u515005323	식품	p2501674	59.18	V
2022-06-04 23:59:44	u557663879	식품	p1005008	84.69	V

32770 rows × 5 columns

3. 시계열 배열 생성(date_range)과 주기

pandas date_range

특정 주기의 DatetimeIndex를 만드는 함수

start

배열의 시작

end

배열의 끝

periods (int, optional)

생성할 배열의 숫자

freq (주기 / 기본값 'D')

생성할 데이터의 주기

```
In [10]: # 실습 준비 코드
import pandas as pd
```

```
In [11]: '''
`pandas.date_range()` 함수는 일련의 날짜(datetime)를 생성하는 데 사용되는 강력한 기능입니다.
이 함수는 주어진 범위 내에서 일정한 간격으로 날짜를 생성하거나,
특정 빈도(freq)로 날짜를 생성할 수 있습니다.

`date_range()` 함수의 주요 매개변수는 다음과 같습니다:

- start: 생성할 범위의 시작 날짜를 지정합니다.
- end: 생성할 범위의 끝 날짜를 지정합니다.
- periods: 생성할 날짜의 총 개수를 지정합니다.
- freq: 생성할 날짜의 빈도를 지정합니다. 기본값은 'D'로 하루를 의미합니다.
  다른 가능한 값으로는 'H' (시간), 'M' (분), 'Y' (연도) 등이 있습니다.
```

예를 들어, 2024년 1월 1일부터 2024년 1월 10일까지의 날짜를 생성하려면 다음과 같이 사용할 수 있습니다:

```
python
import pandas as pd

dates = pd.date_range(start='2024-01-01', end='2024-01-10')
print(dates)
```

출력 결과는 다음과 같습니다:

```
DatetimeIndex(['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04',
               '2024-01-05', '2024-01-06', '2024-01-07', '2024-01-08',
               '2024-01-09', '2024-01-10'],
              dtype='datetime64[ns]', freq='D')
```

이렇게 하면 2024년 1월 1일부터 2024년 1월 10일까지의 날짜를 포함하는 DatetimeIndex가 생성됩니다.

...

`freq` 매개변수를 사용하여 특정 빈도로 날짜를 생성하는 예제를 살펴보겠습니다.
다음은 매월 마지막 평일을 포함하는 2024년의 날짜를 생성하는 예제입니다:

```
python
import pandas as pd

dates = pd.date_range(start='2024-01-01', end='2024-12-31', freq='BM')
print(dates)
```

여기서:

- `start` 매개변수는 2024년 1월 1일을 나타냅니다.
- `end` 매개변수는 2024년 12월 31일을 나타냅니다.
- `freq` 매개변수는 'BM'으로 설정되어 있습니다.
이는 'Business Month End'를 의미하며, 매월 마지막 평일을 생성합니다.

출력 결과는 다음과 같습니다:

```
DatetimeIndex(['2024-01-31', '2024-02-29', '2024-03-29', '2024-04-30',
               '2024-05-31', '2024-06-28', '2024-07-31', '2024-08-30',
               '2024-09-30', '2024-10-31', '2024-11-29', '2024-12-31'],
              dtype='datetime64[ns]', freq='BM')
```

이렇게 하면 2024년의 각 월의 마지막 평일을 포함하는 DatetimeIndex가 생성됩니다.

...

```
# 시작일과 끝일 사이의 모든 날짜
pd.date_range('2022-01-03', '2022-01-10')
```

```
Out[11]: DatetimeIndex(['2022-01-03', '2022-01-04', '2022-01-05', '2022-01-06',
                    '2022-01-07', '2022-01-08', '2022-01-09', '2022-01-10'],
                    dtype='datetime64[ns]', freq='D')
```

```
In [ ]: # 시작일부터 지정된 개수의 배열 (날짜)
pd.date_range('2022-01-03', periods=3)
```

```
Out[ ]: DatetimeIndex(['2022-01-03', '2022-01-04', '2022-01-05'], dtype='datetime64[ns]', freq='D')
```

```
In [ ]: # 결과가 index인 시리즈 만들기
idx = pd.date_range('2022-01-03', periods=3)
pd.Series([0, 1, 2], index=idx)
```

```
Out[ ]: 2022-01-03    0
        2022-01-04    1
        2022-01-05    2
        Freq: D, dtype: int64
```

```
In [ ]: # DatetimeIndex이지만 시리즈로 만들 수 있고 데이터프레임의 열로 만들 수도 있다
pd.Series(pd.date_range('2022-01-03', periods=3))
```

```
Out[ ]: 0    2022-01-03
        1    2022-01-04
        2    2022-01-05
        dtype: datetime64[ns]
```

```
In [7]: '''
        이 외의 주기가 필요하면 링크를 참조 :
        https://pandas.pydata.org/docs/user_guide/timeseries.html#timeseries-offset-aliases
        '''

        Image('3.png')
```

Out[7]: 주기

문자	timestamp
Y	각 해의 마지막 날
Q	각 분기의 마지막 날
M	각 달의 마지막 날
W	각 주의 일요일
D	일
H	시간
T	분
s	초
B	주말이 아닌 평일
YS, QS, MS	각 해, 분기, 달의 첫 날
W-MON, W-TUE 등	각 주의 해당 요일
Q-FEB	2월이 마지막 분기인 기준일 때의 분기 마지막 날

```
In [ ]: # 주기 바꾸기 (월의 마지막날)
pd.date_range('2022-01-03', periods=3, freq='M')
```

```
Out[ ]: DatetimeIndex(['2022-01-31', '2022-02-28', '2022-03-31'], dtype='datetime64[ns]', freq='M')
```

```
In [ ]: # 주기 바꾸기 (월의 첫날)
pd.date_range('2022-01-03', periods=3, freq='MS')
```

```
Out[ ]: DatetimeIndex(['2022-02-01', '2022-03-01', '2022-04-01'], dtype='datetime64[ns]', freq='MS')
```

```
In [ ]: # 주기 바꾸기 (2개월 주기의 마지막날)
pd.date_range('2022-01-03', periods=3, freq='2M')
```

```
Out[ ]: DatetimeIndex(['2022-01-31', '2022-03-31', '2022-05-31'], dtype='datetime64[ns]', freq='2M')
```

```
In [ ]: # 주기 바꾸기 (년도별 마지막날)
pd.date_range('2022-01-03', periods=3, freq='Y')

Out[ ]: DatetimeIndex(['2022-12-31', '2023-12-31', '2024-12-31'], dtype='datetime64[ns]', freq='A-DEC')

In [ ]: # 주기 바꾸기 (10분)
pd.date_range('2022-01-03', periods=10, freq='10T')

Out[ ]: DatetimeIndex(['2022-01-03 00:00:00', '2022-01-03 00:10:00',
                        '2022-01-03 00:20:00', '2022-01-03 00:30:00',
                        '2022-01-03 00:40:00', '2022-01-03 00:50:00',
                        '2022-01-03 01:00:00', '2022-01-03 01:10:00',
                        '2022-01-03 01:20:00', '2022-01-03 01:30:00'],
                        dtype='datetime64[ns]', freq='10T')
```

4. dt 접근자

```
In [8]: Image('4.png')
```

Out[8]: **dt** 접근자와 사용해서 **datetime**의 특정 데이터를 가져오는 속성과 함수들

속성 또는 함수	설명	자료형
year	연도	int
quarter	분기 (ex 2021Q3에 해당하면 3)	int
month	월	int
day	일	int
hour	시	int
minute	분	int
second	초	int
weekday	요일(월요일이 0, 일요일이 6)	int
dayofyear	연중 몇 번째 날 (ex 2월 5일이면 36)	int
days_in_month	소속 달의 총 일수(ex 1월이면 31)	int
date	날짜 부분만 (ex 2022-01-01), 시리즈에 사용	str
time	시간 부분만 (ex 02:30:00), 시리즈에 사용	str
day_name()	요일의 영문명(ex Saturday)	str
month_name()	월의 영문명 (ex January)	str
isocalendar()	연도, 주, 요일을 정수인 데이터프레임으로 반환	int
normalize()	시간을 00:00:00으로 정규화 시켜 날짜만 표기	datetime

```
In [ ]: # 실습 준비 코드
import pandas as pd
idx = pd.date_range('2022-01-03', periods=6, freq='430 9H 10T')
df1 = pd.DataFrame(idx, columns=['date'])
df2 = pd.DataFrame([20, 20, 30, 20, 40, 10], index=idx, columns=['sales'])
```

```
In [ ]: # 실습에 쓰일 df1
df1
```


Out[]: **date**

```
0 2022-01-03 00:00:00
1 2022-02-15 09:10:00
2 2022-03-30 18:20:00
3 2022-05-13 03:30:00
4 2022-06-25 12:40:00
5 2022-08-07 21:50:00
```

In []: *# df1의 date열의 날짜만 추출*
df1['date'].dt.day

Out[]:

```
0    3
1   15
2   30
3   13
4   25
5    7
Name: date, dtype: int64
```

In []: df1['date'].dt.year *# 연도*
df1['date'].dt.quarter *# 쿼터*
df1['date'].dt.month *# 월*
df1['date'].dt.day *# 일*
df1['date'].dt.hour *# 시각*
df1['date'].dt.minute *# 분*
df1['date'].dt.second *# 초*

Out[]:

```
0    0
1    0
2    0
3    0
4    0
5    0
Name: date, dtype: int64
```

In []: *# df1의 date열의 시간만 추출 (결과는 object)*
df1['date'].dt.time

```
Out[ ]:
0    00:00:00
1    09:10:00
2    18:20:00
3    03:30:00
4    12:40:00
5    21:50:00
Name: date, dtype: object
```

```
In [ ]: # df1의 date열의 날짜만 추출 (결과는 object)
df1['date'].dt.date
```

```
Out[ ]:
0    2022-01-03
1    2022-02-15
2    2022-03-30
3    2022-05-13
4    2022-06-25
5    2022-08-07
Name: date, dtype: object
```

```
In [ ]: '''
`df1['date'].dt.normalize()`는 DataFrame의 'date' 열에서 날짜를
정규화(normalize)하는 방법입니다.
이는 시간 구성 요소를 제거하고 날짜만 포함하는 새로운 Series를 생성합니다.
이렇게 하면 모든 날짜가 같은 날짜지만 다른 시간으로 표시될 때 유용합니다.

예를 들어, 다음은 DataFrame에서 'date' 열의 시간 구성 요소를 제거하여
정규화하는 방법을 보여줍니다:

python
import pandas as pd

# 예제 DataFrame 생성
df1 = pd.DataFrame({'date': ['2024-04-27 08:00:00', '2024-04-28 12:30:00', '2024-04-29 15:45:00']})

# 'date' 열에서 날짜를 정규화
normalized_dates = df1['date'].dt.normalize()

print(normalized_dates)

출력 결과는 다음과 같습니다:

0    2024-04-27
1    2024-04-28
```

```
2    2024-04-29
Name: date, dtype: datetime64[ns]
```

이제 'date' 열의 시간 구성 요소가 제거되고, 모든 날짜가 같은 날짜지만 시간은 00:00:00으로 설정된 것을 확인할 수 있습니다.

```
'''
# 날짜만 표시 (datetime의 시간을 00:00:00으로 만든다)
df1['date'].dt.normalize()
```

```
Out[ ]:
0    2022-01-03
1    2022-02-14
2    2022-03-28
3    2022-05-10
4    2022-06-21
5    2022-08-02
Name: date, dtype: datetime64[ns]
```

```
In [ ]: '''
`df1['date'].dt.isocalendar()`는 DataFrame의 'date' 열에서
ISO 주차 및 요일 정보를 추출하는 메서드입니다.
ISO 주차 및 요일 정보는 ISO 8601 표준에 따라
주어진 날짜의 연도, 주차 번호 및 요일을 튜플 형태로 반환합니다.

이 메서드는 DataFrame의 datetime 열에서 연도, 주차 및 요일 정보를 추출할 때 유용합니다.

예를 들어, 다음은 DataFrame에서 'date' 열의 각 날짜에 대한
ISO 주차 및 요일 정보를 추출하는 방법을 보여줍니다:

python
import pandas as pd

# 예제 DataFrame 생성
df1 = pd.DataFrame({'date': ['2024-04-27', '2024-04-28', '2024-04-29']})

# 'date' 열에서 ISO 주차 및 요일 정보 추출
iso_calendar_info = df1['date'].dt.isocalendar()

print(iso_calendar_info)

출력 결과는 다음과 같습니다:

      year  week  day
0  2024     17    6
```

```
1 2024    17    7
2 2024    17    1
```

여기서:

- 'year' 열은 연도를 나타냅니다.
- 'week' 열은 ISO 주차 번호를 나타냅니다.
- 'day' 열은 ISO 요일 번호를 나타냅니다. (월요일부터 1, 일요일까지 7)
- ...

```
# 연도 주 날짜를 데이터 프레임으로(주로 주를 추출할 때 쓴다)
df1['date'].dt.isocalendar()
```

Out[]:

	year	week	day
--	------	------	-----

0	2022	1	1
---	------	---	---

1	2022	7	1
---	------	---	---

2	2022	13	1
---	------	----	---

3	2022	19	2
---	------	----	---

4	2022	25	2
---	------	----	---

5	2022	31	2
---	------	----	---

In []: # 실습에 쓰일 df2
df2

Out[]:

	sales
--	-------

2022-01-03 00:00:00	20
---------------------	----

2022-02-15 09:10:00	20
---------------------	----

2022-03-30 18:20:00	30
---------------------	----

2022-05-13 03:30:00	20
---------------------	----

2022-06-25 12:40:00	40
---------------------	----

2022-08-07 21:50:00	10
---------------------	----

```
In [ ]: # df2의 index(DatetimeIndex)에서 연도만 추출 (dt접근자가 필요 없다)
df2.index.year
```

```
Out[ ]: Int64Index([2022, 2022, 2022, 2022, 2022, 2022], dtype='int64')
```

```
In [9]: '''
더 많은 지시자를 알고 싶다면 링크를 참조 :

https://docs.python.org/3/library/datetime.html#strftime-and-strptime-behavior
'''

Image('5.png')
```

Out[9]: 지시자 (directive)

지시자	기능	예 (2022-01-01 18:00:30)
%y	숫자 연도의 뒷 두자리	22
%Y	숫자 연도	2022
%m	숫자 월, 0으로 채움	01
%B	문자 월, 영문명	January
%b	문자 월, 영문 약자	Jan
%w	숫자 요일 (0이 일요일)	6
%A	문자 요일, 영문명	Saturday
%a	문자 요일, 영문 약자	Sat
%d	숫자 일, 0으로 채움	01
%H	숫자 시각, 24시 기준, 0으로 채움	18
%I	숫자 시각, 12시 기준, 0으로 채움	06
%p	AM 과 PM의 구분	PM
%M	숫자 분, 0으로 채움	00
%S	숫자 초, 0으로 채움	30

In []: '''

df1['date'].dt.strftime() 메서드는 DataFrame의 datetime 열에서 문자열 포맷을 지정하여 문자열로 변환하는 메서드입니다. 이 메서드를 사용하면 날짜를 원하는 형식으로 문자열로 변환할 수 있습니다.

메서드의 인자로써 날짜 및 시간 포맷 지정 문자열을 전달합니다.
이 문자열은 날짜 및 시간 요소에 대한 지정된 포맷을 지정하는 데 사용됩니다.

포맷 코드는 다음과 같습니다:
이 목록은 Python의 `strftime()` 및 `strptime()` 함수에서 사용되는 포맷 코드와 동일합니다.

- %Y: 4자리 연도 (예: 2022)

```

- %m: 월 (01부터 12)
- %d: 일 (01부터 31)
- %H: 24시간 형식의 시간 (00부터 23)
- %I: 12시간 형식의 시간 (01부터 12)
- %M: 분 (00부터 59)
- %S: 초 (00부터 59)
- %A: 요일의 전체 이름 (예: Monday)
- %a: 요일의 축약된 이름 (예: Mon)
- %B: 월의 전체 이름 (예: January)
- %b: 월의 축약된 이름 (예: Jan)
- %j: 연중 일 수 (001부터 366)
- %U: 연중 주 번호 (00부터 53, 일요일을 시작으로 1주부터 시작)
- %W: 연중 주 번호 (00부터 53, 월요일을 시작으로 1주부터 시작)
- %w: 요일 번호 (0부터 6, 일요일을 0으로)
- %Z: 시간대 (예: PST)
- %z: UTC 오프셋 (+hhmm 형식, -hhmm 형식)
- %%: % 문자
'''

```

```
# 표기형식 바꾸기
```

```
df1['date'].dt.strftime('%y/%m/%d [%a]')
```

```

Out[ ]: 0    22/01/03 Mon
        1    22/02/15 Tue
        2    22/03/30 Wed
        3    22/05/13 Fri
        4    22/06/25 Sat
        5    22/08/07 Sun
Name: date, dtype: object

```

```

In [ ]: # 연도와 합쳐진 월의 데이터를 가져올 때도 strftime을 쓴다
df1['date'].dt.strftime('%Y-%m')

```

```

Out[ ]: 0    2022-01
        1    2022-02
        2    2022-03
        3    2022-05
        4    2022-06
        5    2022-08
Name: date, dtype: object

```

```

In [ ]: '''
        `to_period()` 메서드는 datetime 혹은 timedelta 객체를 Period 객체로 변환하는 메서드입니다.
        이 메서드를 사용하면 일정한 기간을 나타내는 Period 객체로 변환할 수 있습니다.

```

주요 매개변수로는 변환할 기간의 빈도(freq)를 지정하는 `freq`가 있습니다. 이것은 기본적으로 'D'로 하루를 나타내며 빈도 코드는 아래와 같습니다.

- 'D': 일(day)
- 'B': 평일(weekday)
- 'W': 주(week), 주의 시작은 일요일(Sunday)
- 'W-SUN': 주(week), 주의 시작은 일요일(Sunday)
- 'W-MON': 주(week), 주의 시작은 월요일(Monday)
- 'M': 월(month), 각 달의 마지막 날짜를 기준으로 함
- 'MS': 월(month), 각 달의 첫째 날짜를 기준으로 함
- 'BM': 월(month), 평일 기준으로 각 달의 마지막 날짜를 기준으로 함
- 'BMS': 월(month), 평일 기준으로 각 달의 첫째 날짜를 기준으로 함
- 'Q': 분기(quarter), 각 분기의 마지막 날짜를 기준으로 함
- 'Q-DEC': 분기(quarter), 12월을 기준으로 하는 연도 분기
- 'A': 연(year), 각 연도의 마지막 날짜를 기준으로 함
- 'AS': 연(year), 각 연도의 첫째 날짜를 기준으로 함
- 'BQ': 분기(quarter), 평일 기준으로 각 분기의 마지막 날짜를 기준으로 함
- 'BA': 연(year), 평일 기준으로 각 연도의 마지막 날짜를 기준으로 함
- 'BAS': 연(year), 평일 기준으로 각 연도의 첫째 날짜를 기준으로 함
- 'BH': 시간(hour), 평일 기준으로 각 시간의 마지막을 기준으로 함
- 'H': 시간(hour), 각 시간의 마지막을 기준으로 함
- 'T': 분(minute)
- 'S': 초(second)

예를 들어, datetime 객체를 Period 객체로 변환하려면:

```
import pandas as pd

# datetime 객체 생성
date_time = pd.to_datetime('2024-04-27')

# datetime 객체를 Period 객체로 변환
period = date_time.to_period()

print(period)
출력 결과는 다음과 같습니다:

Period('2024-04-27', 'D')
```

이것은 2024년 4월 27일을 나타내는 Period 객체입니다.

`to\_period()` 메서드를 Series나 DataFrame의 datetime 열에 적용하면, 해당 열의 모든 값을 Period 객체로 변환합니다. 이는 시계열 데이터를 일정한 기간으로 요약하거나 집계할 때 유용합니다.

예를 들어, 날짜를 월단위로 요약하려면:

```
import pandas as pd

# 예제 DataFrame 생성
df1 = pd.DataFrame({'date': ['2024-04-27', '2024-05-28', '2024-06-29']})

# 'date' 열의 값을 datetime 형식으로 변환
df1['date'] = pd.to_datetime(df1['date'])

# 변환된 'date' 열의 날짜를 월단위로 변환
monthly_periods = df1['date'].dt.to_period('M')

print(monthly_periods)
```

위 코드에서는 'date' 열의 각 날짜를 월단위로 변환하여 Period 객체로 반환합니다.

```
0    2024-04
1    2024-05
2    2024-06
Name: date, dtype: period[M]
'''
```

연도가 포함된 월을 얻고 싶다면 period 자료형을 사용하는 것이 더 정확하다
df1['date'].dt.to_period('M')

Out[]:

```
0    2022-01
1    2022-02
2    2022-03
3    2022-05
4    2022-06
5    2022-08
Name: date, dtype: period[M]
```

In []: *# 연도가 포함된 쿼터를 얻을 경우도 시계열을 유지하려면 period 자료형을 쓴다*
df1['date'].dt.to_period('Q')

```
Out[ ]:
0    2022Q1
1    2022Q1
2    2022Q1
3    2022Q2
4    2022Q2
5    2022Q3
Name: date, dtype: period[Q-DEC]
```

5. 시계열 데이터의 그룹집계(resample)

```
In [14]: # 실습 준비 코드
import pandas as pd
idx = pd.date_range('2023-01-01', periods=6, freq='200 9H 10T')
df1 = pd.DataFrame(idx, columns=['날짜'])
df1['금액'] = [10000, 20000, 20000, 20000, 30000, 10000]
df1['구분'] = ['입금', '입금', '출금', '입금', '입금', '출금']
df2 = df1.set_index('날짜')
df1
```

```
Out[14]:
```

	날짜	금액	구분
0	2023-01-01 00:00:00	10000	입금
1	2023-01-21 09:10:00	20000	입금
2	2023-02-10 18:20:00	20000	출금
3	2023-03-03 03:30:00	20000	입금
4	2023-03-23 12:40:00	30000	입금
5	2023-04-12 21:50:00	10000	출금

만약 df1을 월별 집계를 한다면 df1의 날짜열에서 월의 데이터를 추출해 `groupby` 를 사용할 수도 있을 것이다

```
In [15]: # df1을 복제한 df로 월별 정보를 담은 월 열을 만든다
df = df1.copy()
df['월'] = df['날짜'].dt.strftime('%Y-%m')
df
```

Out[15]:

	날짜	금액	구분	월
0	2023-01-01 00:00:00	10000	입금	2023-01
1	2023-01-21 09:10:00	20000	입금	2023-01
2	2023-02-10 18:20:00	20000	출금	2023-02
3	2023-03-03 03:30:00	20000	입금	2023-03
4	2023-03-23 12:40:00	30000	입금	2023-03
5	2023-04-12 21:50:00	10000	출금	2023-04

In [16]: `# groupby로 월별 금액을 집계한다`
`df.groupby('월')['금액'].sum()`

Out[16]:

월	
2023-01	30000
2023-02	20000
2023-03	50000
2023-04	10000

Name: 금액, dtype: int64

In [17]: `'''`
 코드는 먼저 '월'과 '구분' 열을 기준으로 그룹화합니다.
 그런 다음 '금액' 열의 값을 합산하여 결과를 반환합니다.
`'''`
`df.groupby(['월', '구분'])['금액'].sum()`

Out[17]:

월	구분	
2023-01	입금	30000
2023-02	출금	20000
2023-03	입금	50000
2023-04	출금	10000

Name: 금액, dtype: int64

In []: `'''`
`unstack()` 메서드는 그룹화된 결과를 재구조화하여
 다중 인덱스를 가진 Series를 DataFrame으로 변환합니다.
 '월'과 '구분'을 인덱스로 하는 DataFrame을 반환합니다.
`'''`

```
# groupby로 월과 구분으로 집계해 unstack으로 피벗테이블처럼 만든다
df.groupby(['월', '구분'])['금액'].sum().unstack()
```

```
Out[ ]:
```

	구분	입금	출금
월			
2023-01		30000.0	NaN
2023-02		NaN	20000.0
2023-03		50000.0	NaN
2023-04		NaN	10000.0

위 방법이 일감으로 떠오르겠지만 판다스에서는 대부분 이런식의 그룹집계를 하지 않는다

판다스에는 그룹집계에 탁월한 `resample` 함수가 있기 때문

`groupby` 자리에 `resample` 을 사용하며 따로 열로 만들 것 없이 `resample` 에 집계할 주기를 지정하면 된다

```
In [4]: '''
`resample()` 함수는 시계열 데이터를 재샘플링하기 위해 사용되는 pandas의 메서드입니다.
이 메서드를 사용하면 기존의 시계열 데이터를 다른 주파수나 주기로 변환할 수 있습니다.
아래는 `resample()` 함수의 주요 매개변수들입니다:

- rule: 재샘플링할 주기를 지정하는 매개변수입니다.
  문자열로 빈도(frequency)를 지정하거나 DateOffset 객체를 전달할 수 있습니다.
  예를 들어, 'M'은 월단위, 'W'는 주단위, 'D'는 일단위를 나타냅니다.
  DateOffset 객체는 임의의 주기를 지정할 수 있습니다.
  자세한 내용은
  https://pandas.pydata.org/docs/reference/api/pandas.date_range.html#pandas.date_range 을 참조하세요.
- how: 재샘플링할 구간의 데이터를 어떻게 집계할지를 지정하는 매개변수입니다.
  기본값은 'mean'으로 평균을 의미하며,
  다른 옵션으로 'sum', 'count', 'min', 'max', 'first', 'last' 등이 있습니다.
- closed: 재샘플링할 구간의 어느 쪽을 닫을지를 지정하는 매개변수입니다.
  기본값은 'right'로 오른쪽(구간의 끝)을 닫습니다.
  다른 옵션으로 'left'가 있습니다.
- label: 각 재샘플링된 구간의 라벨을 어디에 지정할지를 지정하는 매개변수입니다.
  기본값은 'right'로 구간의 끝에 라벨을 지정합니다.
  다른 옵션으로 'left'가 있습니다.
- offset: 재샘플링된 결과의 인덱스를 조정하기 위한 오프셋을 지정하는 매개변수입니다.
'''
```

- base: 시작 날짜를 지정하는 매개변수입니다.
 주기가 월 또는 분기인 경우에만 사용할 수 있습니다.
- on: DataFrame에서 재샘플링할 시간 축으로 사용할 열을 지정하는 매개변수입니다.

이러한 매개변수들을 사용하여 `resample()` 함수를 호출하면 원하는 주기로 시계열 데이터를 재샘플링할 수 있습니다.

```
'''
```

```
'''
```

코드는 다음과 같은 작업을 수행합니다:

1. `df1` 데이터프레임의 '날짜' 열을 기준으로 resample(재샘플링)을 수행합니다.
2. `rule='M'`을 지정하여 월 단위로 재샘플링합니다.
 이는 '월'로 그룹화하고 그룹의 각 월에 대해 작업을 수행한다는 것을 의미합니다.
3. `on='날짜'`를 사용하여 '날짜' 열을 기준으로 재샘플링합니다.
4. `[ '금액' ].sum()`을 사용하여 각 그룹(월)의 '금액'을 합산합니다.
 이렇게 하면 각 월에 해당하는 '금액'의 총합이 계산됩니다.

결과적으로, 각 월별로 '금액' 열의 합계를 계산하여 시리즈 형태로 반환합니다.
만약에 해당 월에 데이터가 없다면 해당 월의 값은 NaN(결측치)가 될 수 있습니다.

```
'''
```

```
# 일자별 집계
```

```
df1.resample(rule='M', on='날짜')['금액'].sum()
```

```
Out[4]: DatetimeIndex(['2023-01-31', '2023-02-28', '2023-03-31', '2023-04-30'], dtype='datetime64[ns]', name='날짜', freq='M')
```

```
In [ ]: # timestamp를 월의 첫날로 바꿔보자
```

```
df1.resample('MS', on='날짜')['금액'].sum()
```

```
Out [ ]: 날짜
2023-01-01    30000
2023-02-01    20000
2023-03-01    50000
2023-04-01    10000
Freq: MS, Name: 금액, dtype: int64
```

```
In [18]: df1.groupby('구분').resample('M', on='날짜')['금액'].sum()
```

```
Out[18]:
```

구분	날짜	
입금	2023-01-31	30000
	2023-02-28	0
	2023-03-31	50000
출금	2023-02-28	20000
	2023-03-31	0
	2023-04-30	10000

Name: 금액, dtype: int64

```
In [ ]:
```

```
'''
그룹화된 데이터를 unstack(level=0)을 사용하여
다중 인덱스를 가진 Series를 DataFrame으로 변환합니다.
이 때, level=0을 지정하여 첫 번째 인덱스를 열로 변환합니다.
이로써 '입금'과 '출금'을 각각의 열로 표현된 DataFrame을 얻을 수 있습니다.
'''
```

```
결과적으로, '입금'과 '출금'을 각각의 열로 하고,
월별로 합산된 '금액'을 표로 나타냅니다.
```

```
'''
# 시계열 resample과 groupby를 함께 사용할 때 (groupby부터 사용한다)
df1.groupby('구분').resample('M', on='날짜')['금액'].sum().unstack(level=0)
```

```
Out[ ]:
```

	구분	입금	출금
날짜			
2023-01-31		30000.0	NaN
2023-02-28		0.0	20000.0
2023-03-31		50000.0	0.0
2023-04-30		NaN	10000.0

```
In [ ]:
```

```
# 실습에 쓰일 df2는 DatetimeIndex이다
df2
```

Out []:

금액 구분

날짜

2023-01-01 00:00:00	10000	입금
2023-01-21 09:10:00	20000	입금
2023-02-10 18:20:00	20000	출금
2023-03-03 03:30:00	20000	입금
2023-03-23 12:40:00	30000	입금
2023-04-12 21:50:00	10000	출금

In []: *# DatetimeIndex일 때는 on의 지정이 필요없다*
 df2.resample('MS')['금액'].sum()

Out []: 날짜
 2023-01-01 30000
 2023-02-01 20000
 2023-03-01 50000
 2023-04-01 10000
 Freq: MS, Name: 금액, dtype: int64

프로젝트의 그룹집계로 프로젝트를 마무리

온라인 쇼핑몰 데이터의 일자별 액션수를 집계해보자

In []: *# 프로젝트 코드*
 import pandas as pd
 pd.options.display.max_rows = 8 *# 판다스 1.4+ 에서 8행까지만 출력*
#url = 'https://raw.githubusercontent.com/panda-kim/csv_files/main/shop_data.csv'
 url = 'shop_data.csv'
csv파일에서 부를때 datetime으로 지정하고 index로 지정하기
 df_ex1 = pd.read_csv(url, parse_dates=['시간'], index_col=0)
 df_ex1

Out []:

	유저코드	카테고리	제품코드	가격	행동
시간					
2022-06-01 00:00:01	u514028527	패션	p1307115	411.59	V
2022-06-01 00:00:13	u550193582	화장품	p12705151	51.22	V
2022-06-01 00:00:13	u551977964	식품	p4100249	583.80	V
2022-06-01 00:00:19	u542985695	식품	p12705928	139.58	V
...	...	...	...	...	...
2022-06-07 23:59:41	u548223253	식품	p4000169	102.94	V
2022-06-07 23:59:47	u566067205	인테리어	p16300035	31.92	V
2022-06-07 23:59:56	u553835872	가전	p1005192	205.67	V
2022-06-07 23:59:59	u518620512	식품	p3600661	306.05	V

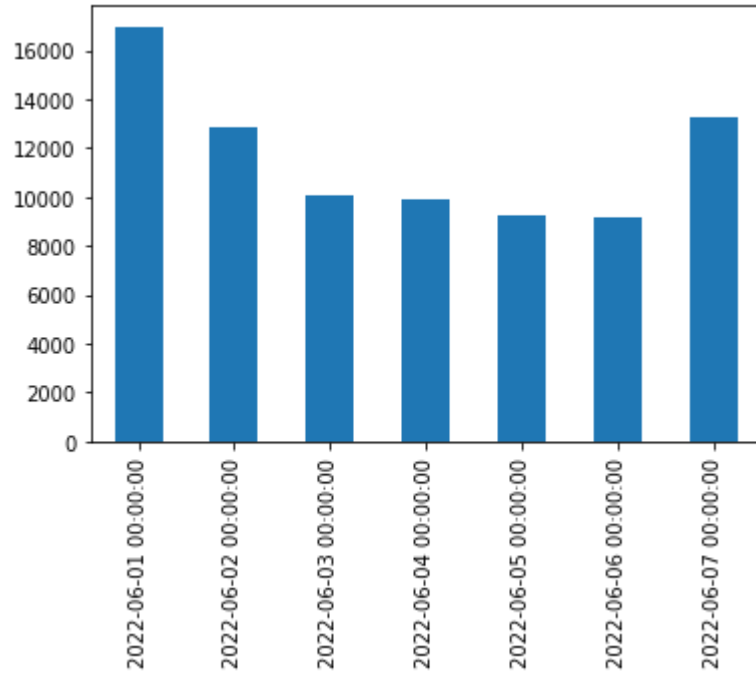
81433 rows × 5 columns

```
In [ ]: # 일자별 총 행동의 횟수 집계
df_ex1.resample('D')['가격'].count()
```

```
Out [ ]: 시간
2022-06-01    16974
2022-06-02    12815
2022-06-03    10063
2022-06-04     9892
2022-06-05     9285
2022-06-06     9141
2022-06-07    13263
Freq: D, Name: 가격, dtype: int64
```

```
In [ ]: # 간단하게 시각화
df_ex1.resample('D')['가격'].count().rename_axis('').plot(kind='bar')
```

```
Out [ ]: <Axes: >
```

프로젝트 데이터의 행동 유형

- V(view) : 페이지 보기
- C(cart) : 장바구니 담기
- B(buy) : 구매

유형별로 횟수를 집계해보자

```
In [ ]: # 일자별 행동 유형별로 횟수를 집계
df_ex1.groupby('행동').resample('D')['가격'].count().unstack(level=0)
```

Out[]:

행동	B	C	V
시간			
2022-06-01	75	91	16808
2022-06-02	72	87	12656
2022-06-03	61	81	9921
2022-06-04	63	63	9766
2022-06-05	46	64	9175
2022-06-06	35	60	9046
2022-06-07	60	93	13110